

JARVIS: Separating Parameterized Learning from Retrieval and Tool-Assisted Gains in a Small Language Model

Aditya Bhatia

Independent Researcher

github.com/Aditya-0167/Jarvis-LLM

Preprint (version 1, October 2026). Not peer reviewed.

Abstract

When a language-model system appears to improve after an intervention, the gain may come from changed parameters, retrieval, tools, or routing, or from a controller that overfits its own evaluation. We present JARVIS, a from-scratch small language-model platform designed to make this attribution problem measurable. JARVIS combines a compact transformer-style language model with persistent checkpoint lineage, public-web acquisition, answer-time retrieval, proposed deterministic-tool interfaces, benchmarking, failure diagnosis, and bounded architecture search. Its archived Generation-10 trajectory began at 6,267,852 tracked parameters and reached 156,144,084 by Generation 10, with 3,520 recorded training steps, 4,643,495 corpus characters, 40 ingested web pages, 103 memory records, and nine accepted architecture changes. These are platform observations, not evidence of general intelligence. Qualitative failures exposed a central attribution problem: the same visible answer can arise through parameter updates, retrieval, tools, or routing, and low training loss can coexist with poor user-visible behavior. We therefore propose a $2 \times 2 \times 2$ factorial attribution design over parameter updates, retrieval, and tools, embedded in a controlled loop in which failures are diagnosed, interventions are budgeted, candidates are evaluated on held-out targets and regression suites, and state is retained only when improvement, retention, resource feasibility, and provenance criteria are met. The paper contributes an auditable experimental design for studying system improvement under constrained compute, together with an explicit separation between executed V6 evidence and the unexecuted V7 hypothesis tests.

1 Introduction

Improving a language-model system is often operationally simple: add data, retrieve information,

add a tool, change the prompt, or update the model. Scientifically, however, these interventions are not interchangeable. If a calculator is invoked, the answer may improve without any change to the neural model; if training loss decreases, the model may still fail to transfer the learned behavior to unseen tasks.

Research agents sharpen this concern. They are increasingly capable of carrying out multiple steps of a scientific workflow: retrieving papers, forming hypotheses, writing code, running experiments, and summarizing findings. Systems such as [Lu et al. \(2024\)](#) demonstrate how a language model can be embedded in a loop that proposes research ideas, implements them, executes experiments, and drafts a report; more recent work on recursively improving research agents makes the compounding-loop question even more explicit ([Srikanth et al., 2026](#)). Related work on iterative self-refinement shows that a model can also improve its outputs through repeated feedback without changing its parameters ([Madaan et al., 2023](#)). These results make a stronger question unavoidable: when a system is placed in a repeated loop, what evidence would establish that the system itself is improving?

The distinction matters because several mechanisms can produce the appearance of improvement. Retrieval can supply information that was absent from the model’s parameters; tools can solve exact subproblems that generation alone would fail; additional training can lower loss by fitting the current corpus; and a controller can optimize repeatedly against a validation set without improving generalization. Continual-learning research similarly emphasizes that gains on new data can coexist with degradation on previously learned capabilities ([Shi et al., 2024](#); [Wu et al., 2024](#)). A useful experiment must therefore measure the causal route to the gain, not only the final answer.

JARVIS was built as a small, inspectable platform for this problem. The neural core is deliber-

ately modest enough to train repeatedly on a single accelerator, while the surrounding system records model state, acquisition provenance, retrieval activity, evaluation results, and architecture proposals. The resulting object is closer to a laboratory platform than to a conventional one-shot language-model training pipeline. The goal is not to assert that a small model has reached broad competence. The goal is to create a setting in which an intervention can be accepted, rejected, or falsified using evidence that survives inspection.

The archived V6 trajectory provides both motivation and constraints. The initial Generation-0 state recorded 6,267,852 parameters; Generation 10 is recorded at 156,144,084 tracked parameters after successive accepted architectural changes, with 3,520 training steps. The system also records 4,643,495 corpus characters, 40 ingested web pages, 103 memory records, and nine accepted architecture changes. At the same time, the trajectory exposed mismatches between optimization and user-visible capability: qualitative tests observed irrelevant retrieval fragments, natural-language arithmetic failures, and GPU-memory exhaustion for some larger candidates. These mixed results are useful because they reveal exactly where a learning claim can become confounded.

This paper makes four contributions. First, it gives an operational definition of parameterized learning that excludes inference-time retrieval and tools from the credit assigned to a parameter update. Second, it frames the controller as a research loop: repeated cycles of propose, implement, run, evaluate, and revise are useful only when each cycle leaves an auditable trail and produces a measurable basis for the next intervention. This connects JARVIS to autonomous-research systems such as The AI Scientist while keeping the stronger claim of recursive self-improvement explicitly as a hypothesis (Lu et al., 2024; Srikanth et al., 2026). Third, it specifies a failure-to-intervention controller spanning knowledge, retrieval, reasoning, tool-routing, optimization, forgetting, and infrastructure failures. Fourth, it defines a reproducibility and governance layer that treats checkpoints, corpus versions, provenance, failed runs, and rejected candidates as scientific evidence. The factorial attribution design is then the principal evaluation mechanism for separating those effects.

We emphasize the boundary of the evidence. The results reported here are observations from the executed V6 trajectory and engineering logs. The

proposed V7 experiments are not presented as completed results. This distinction is deliberate: the central question is not whether a system can narrate self-improvement, but whether repeated research actions produce auditable, transferable gains.

2 Research Positioning

2.1 Continual learning and retention

Continual learning studies adaptation across changing tasks, domains, or time while controlling forgetting. Recent surveys of continual learning for language models organize the problem around continual pre-training, domain adaptation, continual fine-tuning, and evaluation under distribution change (Shi et al., 2024; Wu et al., 2024). JARVIS differs in its emphasis on the controller: the question is not only how a model adapts, but how a system chooses what to change after observing a failure. Retention is consequently treated as a first-class outcome, consistent with the continual-learning literature on catastrophic forgetting (Kirkpatrick et al., 2017), rather than an after-the-fact diagnostic.

2.2 Retrieval and tool augmentation

Retrieval-augmented generation separates external information access from parametric storage (Lewis et al., 2020). Tool-augmented systems similarly extend model capabilities through external operations. Toolformer learns to use tools through self-supervision (Schick et al., 2023), while ReAct interleaves reasoning and action (Yao et al., 2023). JARVIS does not claim novelty in retrieval or tool use. Its experimental contribution is to make these mechanisms separately measurable so that a retrieved fact or deterministic computation is not misclassified as parameterized learning.

2.3 Self-refinement and autonomous research

Self-Refine demonstrates iterative improvement through feedback at inference time (Madaan et al., 2023). The AI Scientist demonstrates a broader loop of idea generation, coding, experimentation, analysis, and paper writing (Lu et al., 2024). These systems motivate recursive loops but also make attribution difficult when multiple mechanisms change together. JARVIS focuses on the complementary methodological problem: how to determine whether a repeated loop improves the underlying model, the routing system, or only the information available at inference time.

2.4 Evaluation as the bottleneck

The central challenge is causal attribution. A controller can appear to improve a model if it repeatedly mines the same validation items, if training data overlaps with evaluation, or if a candidate is given an unequal compute or retrieval budget. Our design therefore separates development, validation, regression, and final-test partitions and fixes intervention budgets before candidates are compared. This reflects the broader lesson from benchmark and continual-learning research that evaluation design is part of the system, not a reporting afterthought. Compute- and scaling-law analyses likewise motivate treating data and compute as explicit experimental variables (Hoffmann et al., 2022; Kaplan et al., 2020).

3 Problem Formulation

Let x be a task, M the parameterized model, R the retrieval subsystem, T the deterministic-tool subsystem, and C the controller. The observable response is

$$y = F(x; M, R, T, C). \quad (1)$$

A change in y does not identify which component caused the change. To measure the components separately, we propose binary indicators $U, R, T \in \{0, 1\}$ for whether a parameter update, retrieval, and tool access are enabled. For a fixed candidate checkpoint and task set, the factorial model is

$$\begin{aligned} y = & \beta_0 + \beta_U U + \beta_R R + \beta_T T \\ & + \beta_{UR} UR + \beta_{UT} UT \\ & + \beta_{RT} RT + \beta_{URT} URT + \epsilon. \end{aligned} \quad (2)$$

The interaction terms are important because retrieval may change which tool is selected, and a parameter update may alter retrieval or tool-routing behavior. We do not assume the coefficients are causal under arbitrary observational data; the design becomes interpretable only when the cells are evaluated under matched tasks, budgets, and randomization or controlled order.

The central learning question is likewise conditional. Let B_{target} denote an unseen target set and G a regression set covering previously validated capabilities. A candidate M' is eligible for retention only when

$$\text{Accept}(M') = \mathbb{I}[\Delta_{\text{target}} \geq \delta, \Delta_{\text{reg}} \geq -\tau, \text{feasible}(M'), \text{prov}(M')]. \quad (3)$$

Here δ and τ are predeclared before the intervention block. Rather than selecting these values after observing a favorable run, we propose calibrating them from pilot variance and a practical minimum effect size, then freezing them for the candidate comparison. The protocol records both the calibration evidence and the final values in the experiment manifest.

4 JARVIS System

JARVIS is organized as explicit interfaces so that each source of capability can be measured independently (Figure 1).

4.1 Neural core

The archived Generation-10 configuration uses a vocabulary of 256 symbols, $d_{\text{model}} = 384$, eight attention heads, 12 layers, seven experts with top- $k = 4$ routing, dropout 0.0, and a block size of 1,280. The tracked parameter count is 156,144,084. The transformer backbone follows the attention formulation of Vaswani et al. (2017), while sparse expert routing follows the mixture-of-experts line represented by Shazeer et al. (2017). This is small relative to contemporary foundation models and is intentionally constrained by single-GPU experimentation.

4.2 Persistent state and lineage

Each generation is represented by a checkpoint and associated machine-readable state. A reproducible record links the parent checkpoint, architecture configuration, training steps, corpus version, benchmark version, random seed, optimizer configuration, and checkpoint hash. The controller keeps rejected candidates and interrupted runs rather than deleting them, because resource failures and negative results define the experimentally feasible search space.

4.3 Information acquisition and retrieval

JARVIS distinguishes batch acquisition from answer-time retrieval. Batch acquisition changes the future training corpus; answer-time retrieval supplies evidence to the generator without changing parameters. Acquisition records source URL, query, timestamp, source domain, content hash, and filtering decision. Retrieval should separately record query formulation, retrieved documents, source diversity, duplicate filtering, and relevance judgments. This provenance is essential because search is not learning: an answer supported only by

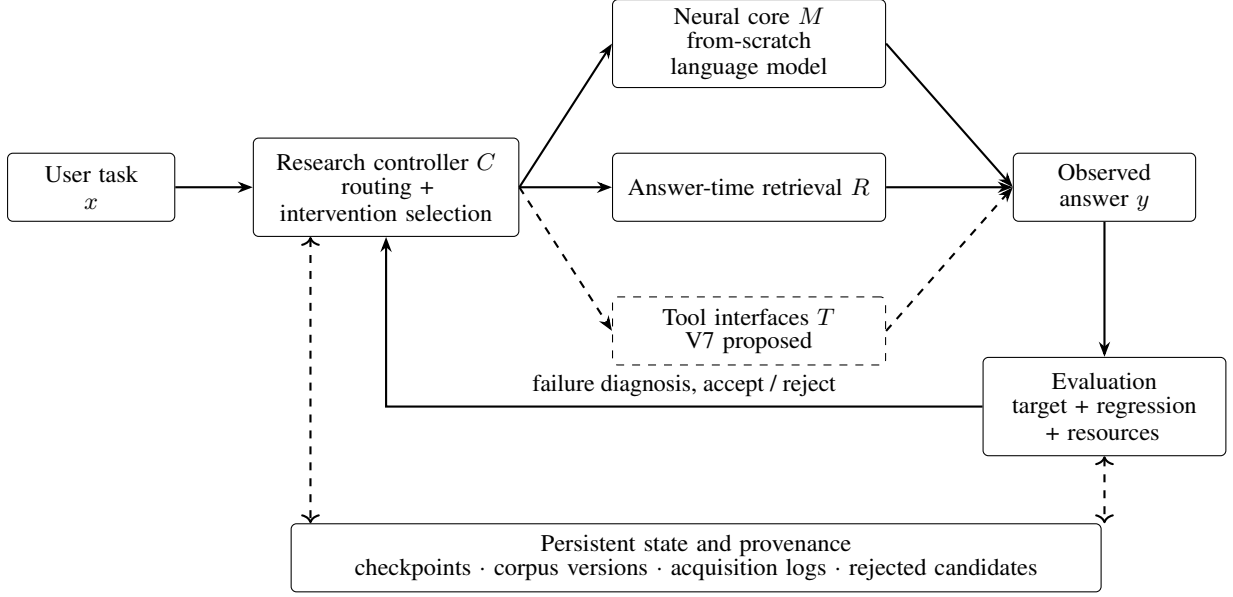


Figure 1: JARVIS separates task routing, neural generation, answer-time information access, proposed tool interfaces, persistent state, and evaluation. This separation is necessary for attribution: the same visible answer can result from different mechanisms and therefore should not receive identical learning credit.

a retrieved document should be credited to information access unless a later parameter-only evaluation demonstrates transfer.

4.4 Tools and routing

Exact operations such as arithmetic or verifiable program execution should be routed to deterministic mechanisms when available. Tool choice is therefore scored independently from final answer correctness. A routing policy can fail even when the model happens to output the correct result, and a routing policy can succeed even if a downstream tool execution fails. This decomposition follows the same scientific principle as retrieval: system capability is a composite property, not a single neural metric.

4.5 Controller

The controller maps observed failures to candidate interventions. The initial taxonomy contains seven classes: knowledge gap, retrieval failure, reasoning failure, tool-routing failure, optimization instability, forgetting, and infrastructure failure. The controller records a probability distribution over causes rather than forcing a single diagnosis. This allows future evaluation of both intervention quality and diagnostic accuracy.

Quantity	Archived value
Generation	10
Tracked parameters	156,144,084
Training steps	3,520
Best evaluation loss	0.1163263097
Corpus characters	4,643,495
Ingested web pages	40
Memory records	103
Accepted architecture changes	9
Hardware	CUDA / T4-class GPU
Checkpoint	generation_000010.pt

Table 1: Observed Generation-10 state. These measurements describe one experimental trajectory and are not normalized performance comparisons.

5 Archived V6 Evidence

The V6 trajectory is useful as a baseline and failure analysis, not as a claim of recursive self-improvement. Table 1 summarizes the state recorded by the archived manuscript.

Three observations motivate the controlled protocol.

Optimization is not capability. The system records substantial reductions in training loss over its trajectory, but qualitative testing did not show a monotonic increase in user-visible capability. A low loss on the current corpus can reflect memorization or distribution-specific optimization rather than transfer to unseen tasks. The manuscript therefore treats training loss as an optimization diagnos-

tic rather than a general capability measure.

Retrieval can improve and simultaneously fail.

The system can return useful public evidence for a factual query while exposing irrelevant learned chunks through the same interface. This is not a paradox; it is evidence that retrieval relevance and answer quality must be scored independently. The failure also illustrates why a retrieval trace belongs in the experiment log.

Architecture search is resource-bounded.

Larger candidate configurations encountered GPU-memory limits on the available accelerator. Such failures are not merely implementation noise. They define the feasible region of the architecture search and motivate hardware-aware screening before expensive candidate runs.

The loop executes repeatedly. A continuous-integration workflow ran bounded research cycles, most of them on a schedule, between 24 and 29 September 2026. The successful runs shown in the workflow history took roughly 7 to 52 minutes each, and the most recent produced a 21.6 GB state artifact. This is evidence that persistent, resumable execution is operationally feasible. It is evidence about infrastructure, not about learning.

Table 2 makes the observed/proposed boundary explicit.

6 From Failure to Intervention

The learning loop begins with a frozen parent rather than an already-modified state (Figure 2). This is important because otherwise a later candidate may inherit undocumented changes and make the attribution chain ambiguous.

6.1 Failure diagnosis

The controller considers a structured distribution over causes. A knowledge-gap diagnosis may trigger targeted public acquisition; a retrieval diagnosis may trigger query rewriting or source filtering; a reasoning failure may trigger worked examples or decomposition; a tool-routing failure may trigger routing-policy updates; optimization failures may trigger a changed schedule; forgetting may trigger replay; and infrastructure failures may trigger resource-aware candidate screening. The controller does not receive credit for the intervention until the candidate is evaluated under the same target and regression suites.

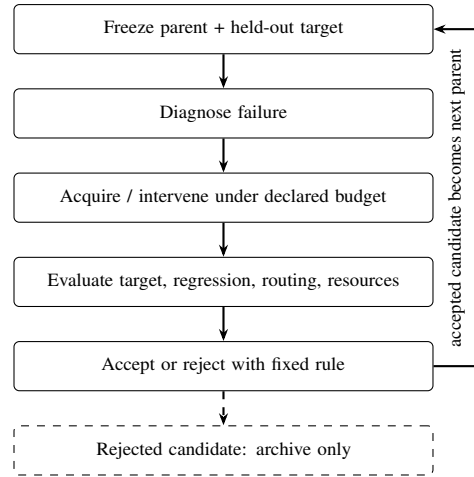


Figure 2: Proposed research loop. Each candidate starts from a frozen parent, receives a declared intervention budget, and is retained only when target improvement, regression, resource feasibility, and provenance criteria are satisfied.

6.2 Targeted learning and replay

The proposed main intervention is targeted learning at approximately fixed model capacity. For a target capability, the controller constructs bounded practice data from the diagnosed gap and optionally mixes in replay examples representing prior validated capabilities. The new-data-to-replay ratio is an explicit experimental variable. This allows comparison of targeted learning with and without retention pressure and prevents a new skill from being declared successful when prior capabilities have regressed.

6.3 Why the controller needs a budget

Recursive systems have a built-in multiple-testing problem: a controller can repeatedly attempt interventions until a validation score looks favorable. To limit this, each intervention block declares training steps, retrieval calls, candidate count, random seeds where feasible, and a wall-clock or memory budget before evaluation. The final-test set remains untouched until the block is complete. This is especially important when the controller itself is the research subject.

7 Controlled V7 Experimental Design

The core experimental comparison is a $2 \times 2 \times 2$ factorial design over parameter updates, retrieval, and tools (Table 3). The design corrects a limitation of a simpler additive attribution story by estimating interaction effects directly.

Evidence status	What JARVIS supports
Observed	Persistent checkpoints, lineage, web acquisition, retrieval, architecture proposals, optimization logs, and reproducibility artifacts were implemented in the V6 trajectory.
Observed failure	Irrelevant retrieval fragments, arithmetic failures, and GPU-memory exhaustion occurred during qualitative/engineering testing.
Not yet established	Error-driven parameter learning on held-out tasks; recursive improvement of research ability; generalization of learned web information; autonomous end-to-end research without human intervention.

Table 2: Claim-status discipline. The proposed V7 protocol is not reported as a completed result.

Cell	Interpretation	U	R	T
1	Model-only	0	0	0
2	Retrieval-only	0	1	0
3	Tool-only	0	0	1
4	Access + tool	0	1	1
5	Parameter-only	1	0	0
6	Parameter + retrieval	1	1	0
7	Parameter + tool	1	0	1
8	Integrated system	1	1	1

Table 3: Proposed factorial attribution matrix. The same task items, budgets, and scoring protocol are used across cells.

This matrix answers several distinct questions. The retrieval-only cell measures information-access benefit without parameter change. The parameter-only cell measures whether a learning intervention changes model behavior without external evidence. The integrated cell measures system-level capability, while the interaction coefficients estimate whether combinations provide gains beyond their isolated effects. Tool-only and tool-enabled cells prevent deterministic computation from being mistaken for neural learning.

7.1 Benchmark composition

The benchmark should prioritize tasks for which the 156M byte-level model has a measurable chance to produce non-floor behavior while still testing transfer. We propose development, validation, regression, and final-test partitions for arithmetic patterns, science concepts, reading comprehension, code reasoning, general factual knowledge, and current-information QA. Current-information items should be time-stamped and evaluated against evidence available under the specified retrieval budget. The final test must remain inaccessible to acquisition and controller tuning.

The floor-effect concern is handled in two ways. First, task families are pilot-screened for non-degenerate baseline behavior rather than selected because they sound ambitious. Second, the study

can include a small external model testbed under the same controller to examine whether observed intervention effects depend on the unusual properties of a from-scratch byte-level model. Such an extension is a generalization study, not part of the archived V6 evidence.

7.2 Statistics

For paired binary correctness outcomes, we propose paired bootstrap intervals and, where sample size permits, exact or McNemar-type tests. Continuous metrics such as latency or loss should be reported with paired effect estimates and dispersion across seeds. Multiple intervention families should be interpreted with their selection process in view. The controller should never repeatedly mine a validation set until a favorable candidate appears. The final test is evaluated once per declared comparison block.

8 Retrieval, Tools, and Attribution

The public-web layer is intentionally split into two paths. Batch acquisition collects versioned material for later learning. Answer-time retrieval supplies current evidence at inference time. The distinction matters for both science and reproducibility: retrieval changes the information available to the generator but does not, by itself, change model parameters. This follows the established RAG framing of external evidence as distinct from parametric storage (Lewis et al., 2020).

Quality control begins at source selection. Relevant first-party technical documentation, primary papers, government and academic sources, and established reporting can receive stronger priors when applicable. Duplicate and low-relevance sources should be filtered. Each acquisition event should record a query, timestamp, URL, source domain, content hash, and filtering result. Retrieval evaluation should report relevance, source diversity, and duplicate rate rather than only top-1 answer success.

Tools create a second attribution problem. If a calculator solves an arithmetic item, the system has demonstrated successful routing and computation, not parameterized arithmetic learning. The benchmark therefore includes tool-enabled and tool-disabled conditions. Exact operations are scored by exact correctness; route selection is scored separately. This decomposition is analogous to the distinction in ReAct between acting with tools and generating a response without tools (Yao et al., 2023).

9 Reproducibility and Governance

JARVIS treats persistence as part of the scientific method. A complete experimental state consists of the checkpoint, model configuration, controller version, source commit, corpus version, benchmark version, seed, optimizer configuration, hardware description, acquisition provenance, and evaluation outputs. Checkpoints are hashed, and rejected candidates remain available for inspection.

The project also distinguishes model generation from controller version. This prevents a change in intervention logic from being confused with a model improvement. The provenance graph should link each training example to the source material from which it was derived and each candidate to the exact data and controller state that created it. This becomes particularly important for web-derived training data, where the same public page may later disappear or change.

Contamination controls operate at multiple levels. Exact string overlap can detect copied evaluation text. Near-duplicate detection can identify lightly transformed examples. Source-level separation prevents a final-test item from being built from the same source instance used for training. Version locking freezes the benchmark and corpus definitions for a study block. Raw benchmark outputs are archived so that changes in scoring logic can be audited.

The system is intentionally bounded. Public-web access is an information pathway, not permission to operate arbitrary external systems. Bounded runs execute on a schedule inside a fixed action space, but a human operator sets the configuration, acceptance margin, benchmark, and state-recovery procedure, and makes the decisions about what to study next. The current research implementation includes batch public-web acquisition and answer-time public-web retrieval; deterministic calculator

routing and sandboxed program execution are specified as V7 extensions rather than reported as completed V6 capabilities. Full autonomous end-to-end research is therefore a proposed V7 property, not a claim about the executed V6 trajectory.

10 Hardware and Resource-Aware Search

The available T4-class accelerator imposes a hard memory envelope. The archived trajectory includes out-of-memory failures for aggressive candidates, demonstrating that architecture search cannot be separated from resource accounting. Candidate evaluation should therefore record parameter count, activation or peak memory, step time, context length, and checkpoint size.

A resource-aware controller can screen candidates before executing them when an empirical memory model is available. Importantly, a candidate that obtains a small validation gain while exceeding the hardware envelope should not become the parent. Feasibility is part of the acceptance criterion rather than an implementation detail.

This resource perspective also changes the experimental objective. Under constrained compute, useful information per unit compute may be more informative than raw parameter growth. The controller can compare interventions using matched step budgets and report the cost of each accepted or rejected change.

11 What the V6 Failure Analysis Changes

The strongest evidence from V6 is methodological. The trajectory demonstrates that persistent experiment state, architectural proposals, public-web acquisition, and evaluation infrastructure can be assembled around a small from-scratch model. It also demonstrates that these mechanisms can fail independently. The existence of a lower loss does not establish better reasoning; the presence of retrieved evidence does not establish learning; and an accepted architecture mutation does not establish generalization.

These observations suggest that the next experiment should not maximize the number of moving parts. It should isolate them. Model-only and retrieval-only conditions distinguish neural behavior from information access. Parameter-only evaluation distinguishes transfer from inference-time assistance. Regression tests distinguish target improvement from forgetting. Tool ablations distinguish routing gains from neural competence. Fixed

budgets distinguish the intervention from hidden compute advantages.

The framework also changes how negative results are treated. A candidate rejected for regression is informative because it shows that the target skill was not acquired without cost. A candidate rejected for memory is informative because it constrains the feasible architecture space. A retrieval candidate rejected for low relevance is informative because it identifies a failure in information acquisition rather than in parameter learning. Preserving these negative paths is as important as preserving the final accepted state.

12 Discussion

The practical lesson is that apparent improvement is a systems property and should be treated as a scientific attribution problem. A system that can search, train, call tools, and modify itself is not automatically a system that learns more effectively over time. The important object is the policy that chooses the next intervention, together with the evidence that the chosen intervention caused a durable gain.

This perspective also clarifies the relationship between model size and improvement. Architecture evolution is one intervention family, not the objective itself. A smaller candidate may be preferable when it fits the resource envelope and preserves prior capabilities. Likewise, retrieval may outperform additional training for current factual questions without changing the model at all. The experimental framework should make these differences explicit rather than collapse them into a single system score.

For research on recursive improvement, auditability is part of the contribution. A loop can only be studied if it leaves a trail: what failed, what the controller believed was wrong, what information it acquired, how much compute it used, what candidate it produced, how the candidate performed on unseen tasks, and why it was accepted or rejected. JARVIS is designed around this trail.

13 Conclusion

JARVIS is a small from-scratch language-model platform for studying error-driven continual learning and its attribution under constrained compute. Its archived Generation-10 state provides a concrete parent checkpoint and a mixed set of engineering observations: persistent lineage and repeated

optimization are operationally feasible, but user-visible capability, retrieval quality, and architecture scalability do not automatically improve with lower loss or larger candidates.

The proposed V7 program turns these observations into a controlled research agenda. A frozen parent is evaluated, failures are diagnosed, interventions are budgeted, candidates are tested under factorial attribution and regression controls, and only candidates meeting predeclared improvement, retention, feasibility, and provenance criteria are retained. The result is not a claim that recursive self-improvement has already been achieved. It is a framework intended to make that claim, if it becomes supportable, falsifiable and reproducible.

Limitations

The current study has several important limitations. First, the archived V6 benchmark was not designed around a frozen, comprehensive capability suite. The reported loss and qualitative failures therefore do not support broad competence claims. Second, the training corpus is small relative to contemporary language-model pretraining, and the byte-level vocabulary creates a very different optimization regime from modern subword-tokenized systems. Third, the documented V6 trajectory contains changing data and evolving system components, which limits clean longitudinal comparisons.

Fourth, retrieval quality depends on external search providers and public-source availability. A future run must preserve provenance rather than assume that repeating a query later will return the same documents. Fifth, hardware dependence constrains which architecture mutations can be evaluated. Sixth, controller evaluation itself can become adaptive if validation items are repeatedly reused. The proposed final-test partition is therefore essential.

The paper makes no claim of frontier-model parity, broad language understanding, or general intelligence. It also does not claim that JARVIS has already demonstrated recursive self-improvement of research ability. The claim is narrower: JARVIS provides an executable systems architecture and a controlled experimental protocol for testing whether repeated error-driven interventions can produce measurable, retained gains in a small from-scratch language model.

Code Availability

The implementation and archived run records are in the public repository listed on the title page. The archived Generation-10 state discussed in this paper is the parent state for the proposed V7 experiments, which have not been run.

References

- Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch, Elena Buchatskaya, Trevor Cai, Eliza Rutherford, Diego de Las Casas, Lisa Anne Hendricks, Johannes Welbl, Aidan Clark, Tom Hennigan, Eric Noland, Katie Millican, George van den Driessche, Bogdan Damoc, Aurelia Guy, Simon Osindero, Karen Simonyan, Erich Elsen, Jack Rae, and Laurent Sifre. 2022. Training compute-optimal large language models. In *Advances in Neural Information Processing Systems* 35.
- Jared Kaplan, Sam McCandlish, Tom Henighan, Tom B. Brown, Benjamin Chess, Rewon Child, Scott Gray, Alec Radford, Jeffrey Wu, and Dario Amodei. 2020. Scaling laws for neural language models. *arXiv preprint arXiv:2001.08361*.
- James Kirkpatrick, Razvan Pascanu, Neil Rabinowitz, Joel Veness, Guillaume Desjardins, Andrei A. Rusu, Kieran Milan, John Quan, Tiago Ramalho, Agnieszka Grabska-Barwinska, Demis Hassabis, Claudia Clopath, Dharshan Kumaran, and Raia Hadsell. 2017. Overcoming catastrophic forgetting in neural networks. *Proceedings of the National Academy of Sciences*, 114(13):3521–3526.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems* 33.
- Chris Lu, Cong Lu, Robert Tjarko Lange, Jakob Foerster, Jeff Clune, and David Ha. 2024. The AI Scientist: Towards fully automated open-ended scientific discovery. *arXiv preprint arXiv:2408.06292*.
- Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegrefe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, Shashank Gupta, Bodhisattwa Prasad Majumder, Katherine Hermann, Sean Welleck, Amir Yazdanbakhsh, and Peter Clark. 2023. Self-refine: Iterative refinement with self-feedback. In *Advances in Neural Information Processing Systems* 36.
- Timo Schick, Jane Dwivedi-Yu, Roberto Dessi, Roberta Raileanu, Maria Lomeli, Luke Zettlemoyer, Nicola Cancedda, and Thomas Scialom. 2023. Toolformer: Language models can teach themselves to use tools. In *Advances in Neural Information Processing Systems* 36.
- Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538*.
- Haizhou Shi, Zihao Xu, Hengyi Wang, Weiyi Qin, Wenyuan Wang, Yibin Wang, Zifeng Wang, Sayna Ebrahimi, and Hao Wang. 2024. Continual learning of large language models: A comprehensive survey. *arXiv preprint arXiv:2404.16789*.
- Dhruv Sriskanth et al. 2026. Recursive self-improvement of AI research agents. *arXiv preprint arXiv:2609.26457*.
- Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems* 30.
- Tongtong Wu, Linhao Luo, Yuan-Fang Li, Shirui Pan, Thuy-Trang Vu, and Gholamreza Haffari. 2024. Continual learning for large language models: A survey. *arXiv preprint arXiv:2402.01364*.
- Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. 2023. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations*.